Week 48

v1.0.0

Table of contents


```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[colab] @ git+https://github.com/au-mbg/fysisk-biokemi.
git#subdirectory=course-utils"
```

---

1 Analysis of a data set obeying Michaelis-Menten kinetics

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from fysisk_biokemi.widgets import DataUploader
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

The kinetics for an enzyme were investigated using the absorbance of the product to calculate the initial velocities for a range of different initial concentrations of the substrate. The data given in the dataset `analys-data-set-obeyin.xlsx` was obtained. Use these data to answer the following questions.

```
uploader = DataUploader()
uploader.display()
```

Run the next cell **after** uploading the file

```
df = uploader.get_dataframe()
display(df)
```

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_2') # Load from package for the solution so it doesn't require to interact.
display(df)
```

	[S]_(mM)	V0_(uM/s)
0	1	166.5
1	1	161.0
2	1	172.0
…	…	…
36	50	478.5
37	50	472.0
38	50	484.0

(a) SI Units

Convert the concentrations of substrate and the velocities to the SI units M and $\mathrm{M} \cdot \mathrm{s}^{-1}$, respectively.

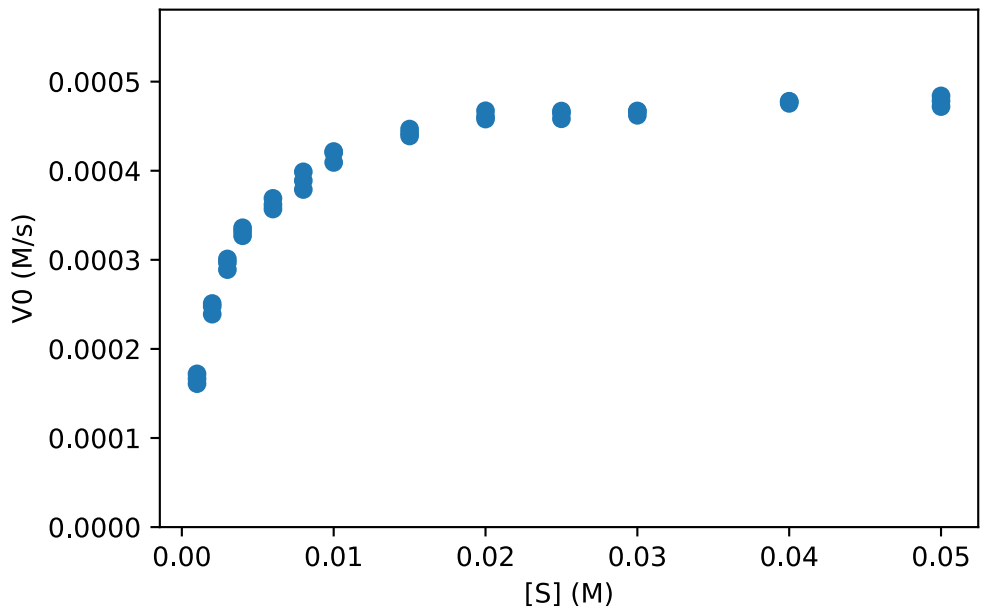
```
## Your task: Add new columns with both properties in proper SI units.
## Use the column names: '[S]_(M)' and 'V0_(M/s)'
df['[S]_(M)'] = df['[S]_(mM)'] * 10**(-3)
df['V0_(M/s)'] = df['V0_(uM/s)'] * 10**(-6)
display(df)
```

	[S]_(mM)	V0_(uM/s)	[S]_(M)	V0_(M/s)
0	1	166.5	0.001	0.000166
1	1	161.0	0.001	0.000161
2	1	172.0	0.001	0.000172
...	...	...	...	...
36	50	478.5	0.050	0.000478
37	50	472.0	0.050	0.000472
38	50	484.0	0.050	0.000484

(b) Plot & estimate

Plot the initial velocities, V_0, as a function of substrate concentrations, $[S]$. Estimate K_M and $V_{\max}$ from this plot.

```
fig, ax = plt.subplots()

## Your task: Plot '[S]_(M)' vs 'V0_(M/s)'.
ax.plot(df['[S]_(M)'], df['V0_(M/s)'], 'o')

## Sets x and y-axis labels:
ax.set_xlabel('[S] (M)')
ax.set_ylabel('V0 (M/s)')

## EXTRA: Sets the y-axis limits.
ax.set_ylim(0, df['V0_(M/s)'].max()*1.2)
```



Put your estimate of K_M and $V_{\max}$ in the cell below:

```
## Task: Put your estimate of the two parameters.
Vmax_estimate = 0.0005
K_M_estimate = 0.005
```

(c) Fit

Now we want to fit using the Michaelis-Menten equation, as per usual when the task is fitting we have to define the function we are fitting with

```
def michaelis_menten(S, K_M, Vmax):
    ## Your task: Implement the Michaelis-Menten equation.
    ## Be careful with parentheses.
    result = (Vmax * S) / (K_M + S)
    return result
```

And then we can follow our usual procedure to make the fit

```
initial_guess = [K_M_estimate, Vmax_estimate]

## Your task: Use curve_fit
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_(M/s)'],
initial_guess)

## Extracts and prints
K_M_fit, Vmax_fit = fitted_parameters
print(Vmax_fit)
print(K_M_fit)
```

```
0.0005007806028375277
0.0020061816467581044
```

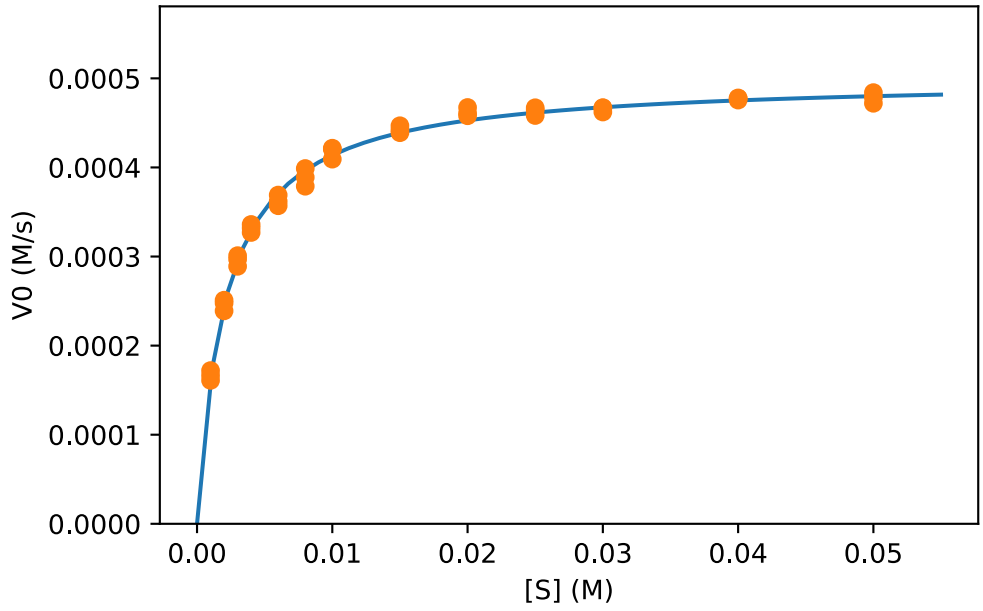
How do these values compare to your estimate?

(d) Plot fit & data

Plot the fit alongside the data.

```
## Your task: Evaluate the fit
S_smooth = np.linspace(0, 0.055)
V0_fit = michaelis_menten(S_smooth, K_M_fit, Vmax_estimate)

## Your task: Make the plot
fig, ax = plt.subplots()

## Your task: Make plot of the fit.
ax.plot(S_smooth, V0_fit)

## This plot the data and sets the options as before.
ax.plot(df['[S]_(M)'], df['V0_(M/s)'], 'o')
```

```
## Sets x and y-axis labels:
ax.set_xlabel('[S] (M)')
ax.set_ylabel('V0 (M/s)')

## EXTRA: Sets the y-axis limits.
ax.set_ylim(0, df['V0_(M/s)'].max()*1.2)
```



(e) k_{cat}

The enzyme concentration was 100 nM use that and your fit to determine k_{cat}.

```
## Your task: Set the enzyme concentration in proper units.
enzyme_concentration = 100 * 10**(-9) # nM

## Your task calculate kcat
kcat = Vmax_fit / enzyme_concentration # (nM/s) / nM  -> 1/s
print(kcat)
```

```
5007.806028375277
```

---

2 The Michaelis Menten equation

Consider a reaction catalyzed by an enzyme obeying the Michaelis-Menten kinetics model.

$$E + S \underset{k_{-1}}{\overset{k_1}{\rightleftharpoons}} ES \overset{k_2}{\rightarrow} E + P$$

$$V_0 = V_{\max} \frac{[S]}{[S] + K_M}$$

(a) Enzyme saturation

Explain what is meant by enzyme saturation.

! Answer

Enzyme saturation simply means the fraction of enzymes in complex with a substrate compared to the total enzyme concentration. Complete saturation occurs when all enzymes is occupied with a substrate molecule.

(b) Enzyme saturation

Calculate the level of enzyme saturation, f_{ES}, at substrate concentrations of

- $[S] = 0.1 \cdot K_M$
- $[S] = 0.5 \cdot K_M$
- $[S] = 1 \cdot K_M$
- $[S] = 10 \cdot K_M$
- $[S] = 100 \cdot K_M$

```
## Your task calculate the degree of enzyme saturation for the prescribed cases
f_ES_1 = 0.1 / (1 + 0.1)
f_ES_2 = 0.5 / (1 + 0.5)
f_ES_3 = 1 / (1 + 1)
f_ES_4 = 10 / (1 + 10)
f_ES_5 = 100 / (1 + 100)

print(f_ES_1, f_ES_2, f_ES_3, f_ES_4, f_ES_5)
```

```
0.09090909090909091 0.3333333333333333 0.5 0.9090909090909091 0.9900990099009901
```

```
## An alternative solution using a function
def enzyme_saturation(x):
    return x / (1 + x)

## Then use the function for each calculation
f_ES_1 = enzyme_saturation(0.1)
f_ES_2 = enzyme_saturation(0.5)
f_ES_3 = enzyme_saturation(1)
f_ES_4 = enzyme_saturation(10)
```

```
f_ES_5 = enzyme_saturation(100)
print(f_ES_1, f_ES_2, f_ES_3, f_ES_4, f_ES_5)
```

```
0.09090909090909091 0.3333333333333333 0.5 0.9090909090909091 0.9900990099009901
```

! Answer

For the Michealis-Menten kinetics model the following holds true

$$[ES] = \frac{([E]_T - [ES])[S]}{K_M}$$

$$[ES] = [E]_T \frac{[S]}{[S] + K_M}$$

$$f_{ES} = \frac{[ES]}{[E]_T} = \frac{[S]}{[S] + K_M}$$

In cases like here where $[S]$ is given in terms of K_M, e.g. $[S] = x \cdot K_M$, we can calculate $f(ES)$ as

$$f_{ES} = \frac{[ES]}{[E]_T} = \frac{xK_M}{xK_M + K_M} = \frac{xK_M}{K_M(1+x)} = \frac{x}{1+x}$$

(c) Compare K_M and K_D.

The Michaelis-Menten constant K_M is defined as:

$$K_M = \frac{k_{-1} + k_2}{k_1}$$

In a specific reaction, the following rate constants were determined:

- $k_1 = 7 \cdot 10^7 \cdot \mathrm{M}^{-1} \cdot \mathrm{s}^{-1}$,
- $k_{-1} = 8 \cdot 10^5\ \mathrm{s}^{-1}$
- $k_2 = 10^3\ \mathrm{s}^{-1}$

Calculate the values of K_M and K_D (The dissociation constant of the enzyme:substrate complex).

Does K_M approximate the dissociation constant for the ES complex in this case, and how can this be estimated directly from the values of the microscopic rate constants?

```
k1 = 7 * 10**7
k2 = 10**3
k_m_1 = 8 * 10**5

K_M = (k_m_1 + k2) / k1
K_D = k_m_1 / k1
```

```
print(K_M)
print(K_D)
```

```
0.011442857142857144
0.011428571428571429
```

! Answer

Yes, since $k_{-1} \gg k_2$ it follows that K_M and K_D for the ES complex are almost identical.

(d) Calculate k_2

Assume that a solution of an enzyme at a concentration of $1 \cdot 10^{-7}$ M with a substrate concentration of $[S] = 100 \cdot K_M$ has $V_0 = 10^{-4}\,\mathrm{M} \cdot \mathrm{s}^{-1}$

Given this information and with your answers to (b) and (c) in mind, approximate the constant k_2. What is this constant also called?

```
## Your task: Assign known values
V0 = 1 * 10**(-4) # M / s
E_tot = 1 * 10**(-7) # M

## Your task: Once you have figured out how to calculate k2, use Python to calculate it.
Vmax = V0 / f_ES_5
k2 = Vmax / E_tot
print(k2)
```

```
1010.0000000000001
```

! Answer

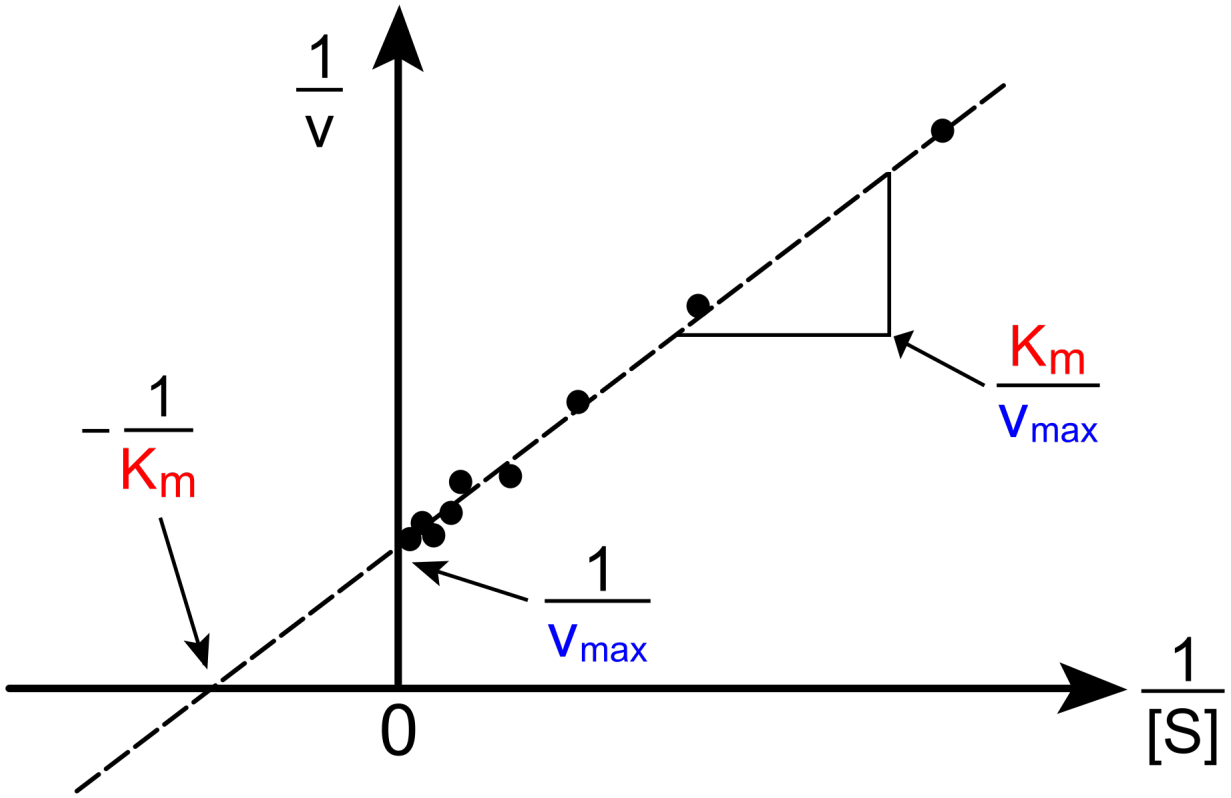
This constant is also known as k_{cat}.

(e) Lineweaver-Burk plot

The values of $V_{\max}$ and K_M have historically been determined from a Lineweaver-Burk plot.

How does the x- and y-intercepts in a Lineweaver-Burk plot relate to $V_{\max}$ and K_M?

! Answer



3 Enzyme inhibitors (I)

```
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit
import numpy as np
pd.set_option('display.max_rows', 6)
```

An enzyme obeying the Michaelis-Menten kinetics model was tested for substrate conversion in the absence and presence of an inhibitor, called inhibitor1 at a concentration of $[\mathrm{I}] = 2.5 \cdot 10^{-3}$ M. The data set is contained in the file `enzyme-inhib-i.xlsx`. Using this data a researcher wanted to determine the type of inhibition.

Start by loading the dataset

```
from fysisk_biokemi.widgets import DataUploader
from IPython.display import display
uploader = DataUploader()
uploader.display()
```

Run the next cell **after** uploading the file

```
df = uploader.get_dataframe()
display(df)
```

```
from IPython.display import display
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_6') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[S]_(mM)	V0_no_inhib_(uM/s)	V0_inhib_(uM/s)
0	1	15.2	5.2
1	1	15.1	5.2
2	1	14.5	5.0
…	…	…	…
36	50	60.7	50.5
37	50	61.4	50.5
38	50	60.8	52.0

(a) Convert units

Convert the concentrations of substrate and the initial velocities to units given in M and $\mathrm{M} \cdot \mathrm{s}^{-1}$, respectively.

```
## Your task: Make new columns with the properties in SI units.
df['[S]_(M)'] = df['[S]_(mM)'] * 10**(-3)
```

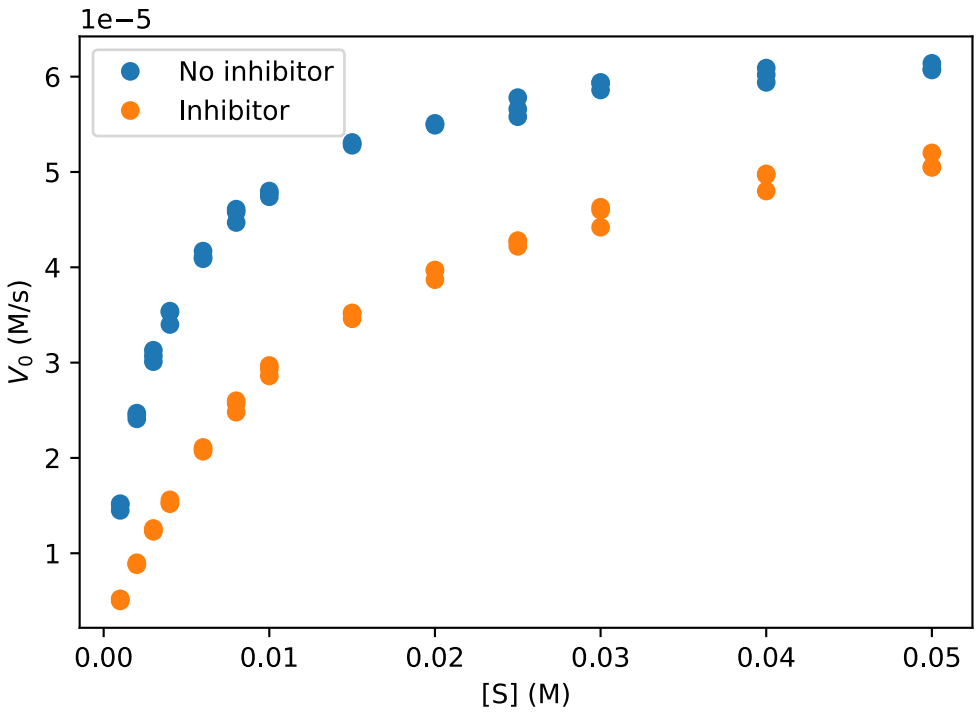
```
df['V0_no_inhib_(M/s)'] = df['V0_no_inhib_(uM/s)'] * 10**(-6)
df['V0_inhib_(M/s)'] = df['V0_inhib_(uM/s)'] * 10**(-6)
display(df)
```

	[S]_(mM)	V0_no_inhib_(uM/s)	V0_inhib_(uM/s)	[S]_(M)	V0_no_inhib_(M/s)	V0_inhib_(M/s)
0	1	15.2	5.2	0.001	0.000015	0.000005
1	1	15.1	5.2	0.001	0.000015	0.000005
2	1	14.5	5.0	0.001	0.000015	0.000005
...	...	...	...	...	...	...
36	50	60.7	50.5	0.050	0.000061	0.000051
37	50	61.4	50.5	0.050	0.000061	0.000051
38	50	60.8	52.0	0.050	0.000061	0.000052

(b) Plot

Plot the initial velocities of both experiments as a function of substrate concentration in one plot. Estimate K_M and $V_{\max}$ in the presence and absence of inhibitor from the plot.

```
fig, ax = plt.subplots(figsize=(6, 4))

## Your task: Plot the initial velocities for both experiments.
ax.plot(df['[S]_(M)'], df['V0_no_inhib_(M/s)'], 'o', label='No inhibitor')
ax.plot(df['[S]_(M)'], df['V0_inhib_(M/s)'], 'o', label='Inhibitor')

## Sets the x and y-axis labels and shows the legend.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)')
ax.legend()
```



Use the plots to estimate K_M and $V_{\max}$ and assign in the cell below

```
## Your task: Estimate parameters without inhibitor.
K_M_no_inhib_estimate = 0.0025 # Unit: M
Vmax_no_inhib_estimate = 6 * 10**(-5) # Unit: M/s

## Your task: Estimate parameters with inhibitor.
K_M_inhib_estimate = 0.01 # Unit: M
Vmax_inhib_estimate = 6 * 10**(-5) # Unit: M/s
```

(c) Fit

The researcher wanted to determine $V_{\max}$ and K_M values for both experiments in order to correctly conclude on the type of inhibitor.

Determine $V_{\max}$ and K_M by fitting.

Start writing the function to fit with

```
## Your task: Implement the Michealis-Menten equation as a function
def michaelis_menten(S, Vmax, Km):
    return (Vmax * S) / (S + Km)
```

And use the `curve_fit`-function to find the parameters

```
## Your task: Make a fit to the Michealies-Menten equation for the data without the
inhibitor.
initial_guess = [Vmax_no_inhib_estimate, K_M_no_inhib_estimate]
```

```
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_no_inhib_(M/
s)'], initial_guess)
Vmax_fit_no_inhib, K_M_fit_no_inhib = fitted_parameters

## Your task: Make a fit to the Michealies-Menten equation for the data the inhibitor.
initial_guess = [Vmax_inhib_estimate, K_M_inhib_estimate]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_inhib_(M/s)'],
initial_guess)
Vmax_fit_inhib, K_M_fit_inhib = fitted_parameters

## Printing
print('Without inhibitor:')
print('    V_max:', Vmax_fit_no_inhib)
print('    K_M:', K_M_fit_no_inhib)
print('With inhibitor:')
print('    V_max:', Vmax_fit_inhib)
print('    K_M:', K_M_fit_inhib)
```

```
Without inhibitor:
    V_max: 6.495499633641784e-05
    K_M: 0.003410851437470283
With inhibitor:
    V_max: 6.3730854803554e-05
    K_M: 0.01220088316450348
```

Which of the kinetic parameters changes substantively in the presence of the inhibitor

! Answer

K_M

(d) Analyze the fit

As per usual, we need to plot the fit to confirm that that it has been done successfully.

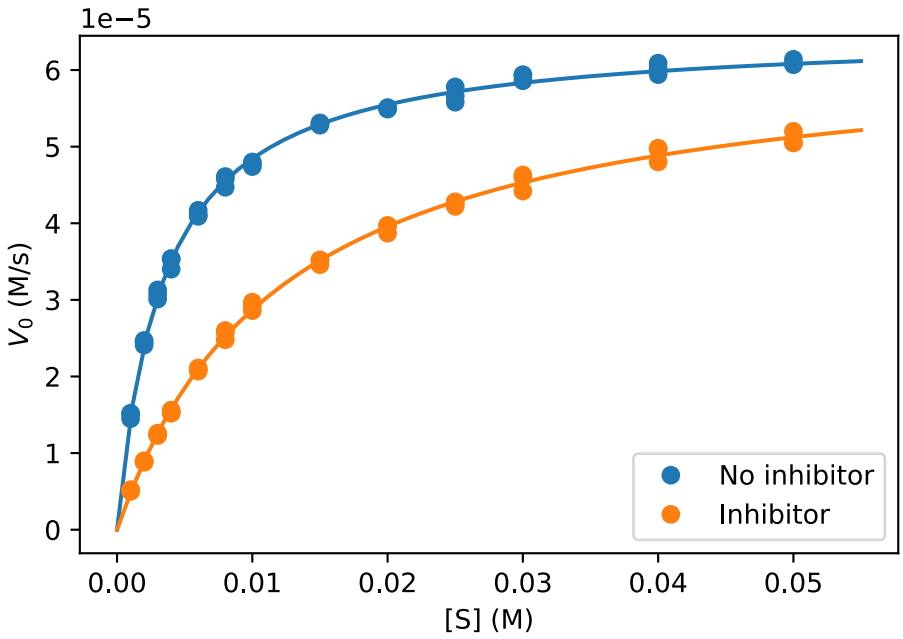
We start by evaluating the fit for each set of parameters - those with and those without the inhibitor.

```
## Your task: Evaluate the fits for both cases
S_smooth = np.linspace(0.0, 0.055)
V0_no_inhib_model = michaelis_menten(S_smooth, Vmax_fit_no_inhib, K_M_fit_no_inhib)
V0_inhib_model = michaelis_menten(S_smooth, Vmax_fit_inhib, K_M_fit_inhib)
```

Copy your code from above and then we plot the fit along with the data,

```
fig, ax = plt.subplots()

## Your task: Plot the fits:
ax.plot(S_smooth, V0_no_inhib_model)
ax.plot(S_smooth, V0_inhib_model)
```

```
## Plots the data:
ax.plot(df['[S]_(M)'], df['V0_no_inhib_(M/s)'], 'o', label='No inhibitor', color='C0')
ax.plot(df['[S]_(M)'], df['V0_inhib_(M/s)'], 'o', label='Inhibitor', color='C1')

## Sets the x and y-axis labels and shows the legend.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)')
ax.legend()
```



(e) Inhibitor type

What type of inhibitor is inhibitor1?

! Answer

As $V_{\max}$ is approximately the same but K_M is significantly increased with the inhibitor that suggests a **competitive** inhibitor.

(f) K_i

The experiment was done in the presence of 2.5 mM of the inhibitor. What is the K_i for this inhibitor?

```
## Your task: Set the inhibitor concentration in M.
I_concentration = 2.5e-3  # M

## Your task: Calculate K_i from the fitted K_M values
K_i = I_concentration / ((K_M_fit_inhib / K_M_fit_no_inhib) - 1)
```

```
print(K_i)
```

```
0.0009700907640015 74
```

---

4 Enzyme inhibitors (II)

```
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit
import numpy as np
from fysisk_biokemi.widgets import DataUploader
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

Two enzyme inhibitors were identified as part of a drug discovery program. To characterize the mechanism of action, the reaction kinetics were analysed for the enzyme alone and in the presence of 5 μM of each of the inhibitors as a function of substrate concentration resulting in the data in the file `enzyme-inhib-ii.xlsx`

Load the dataset

```
uploader = DataUploader()
uploader.display()
```

Run the next cell **after** uploading the file

```
df = uploader.get_dataframe()
display(df)
```

```
from IPython.display import display
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_7') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[S]_(uM)	enz_(nM/s)	inhibitor2_(nM/s)	inhibitor3_(nM/s)
0	0.2	0.070	0.031	0.021
1	0.2	0.067	0.029	0.022
2	0.2	0.071	0.029	0.025
...	...	...	...	...
30	20.0	0.312	0.040	0.119
31	20.0	0.309	0.036	0.113
32	20.0	0.314	0.045	0.120

(a) Convert & plot

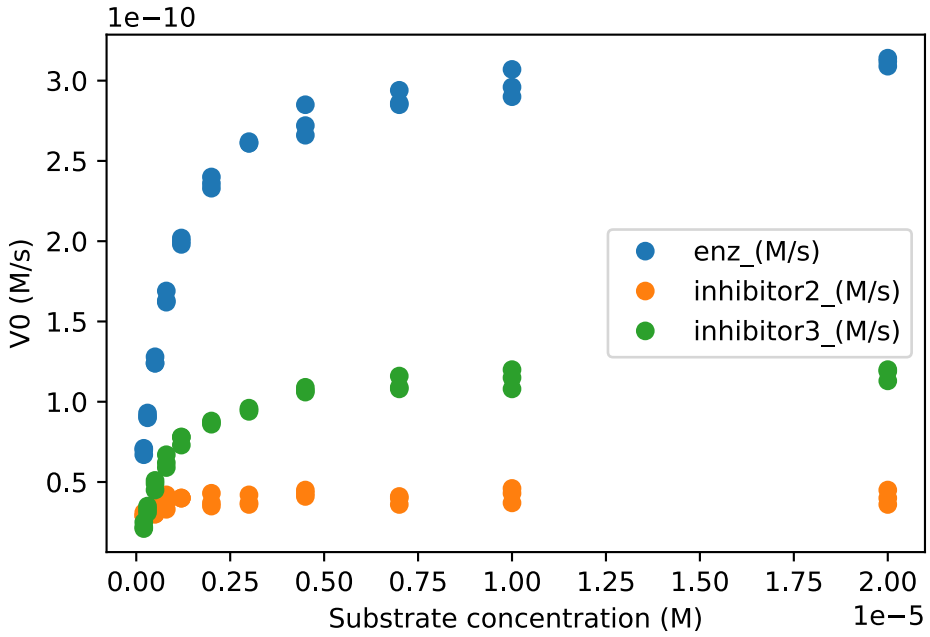
Convert the measurements to SI units and plot the initial reaction velocity versus substrate concentration for all three reactions.

Start by converting the units

```
## Your task: Add new columns with the properties in the correct SI units.
## There are 4 properties, so you should add 4 columns.
df['[S]_(M)'] = df['[S]_(uM)'] * 10**(-6)
df['enz_(M/s)'] = df['enz_(nM/s)'] * 10**(-9)
df['inhibitor2_(M/s)'] = df['inhibitor2_(nM/s)'] * 10**(-9)
df['inhibitor3_(M/s)'] = df['inhibitor3_(nM/s)'] * 10**(-9)
```

And then plot to visualize the data

```
fig, ax = plt.subplots()

## Your task plot each data vs. [S]_(M).
ax.plot(df['[S]_(M)'], df['enz_(M/s)'], 'o', label='enz_(M/s)')
ax.plot(df['[S]_(M)'], df['inhibitor2_(M/s)'], 'o', label='inhibitor2_(M/s)')
ax.plot(df['[S]_(M)'], df['inhibitor3_(M/s)'], 'o', label='inhibitor3_(M/s)')

## Adds legend and axis labels.
ax.legend()
ax.set_xlabel('Substrate concentration (M)')
ax.set_ylabel('V0 (M/s)')
plt.show()
```



(b) Estimate parameters

Based on the appearance of these plots:

- Which dataset would have the highest $V_{\max}$, which the lowest?
- What about K_M?

Based on your answers,

- Can you determine the type of inhibition?
- Alternatively, can you exclude some mechanisms?

! Answer

$V_{\max}$ changes by a lot, with little or no change in K_M. This rules out competetive inhibition.

(c) Fit

As always, we need the function we're fitting with

```
def michaelis_menten(S, Vmax, Km):
    return Vmax * S / (S + Km)
```

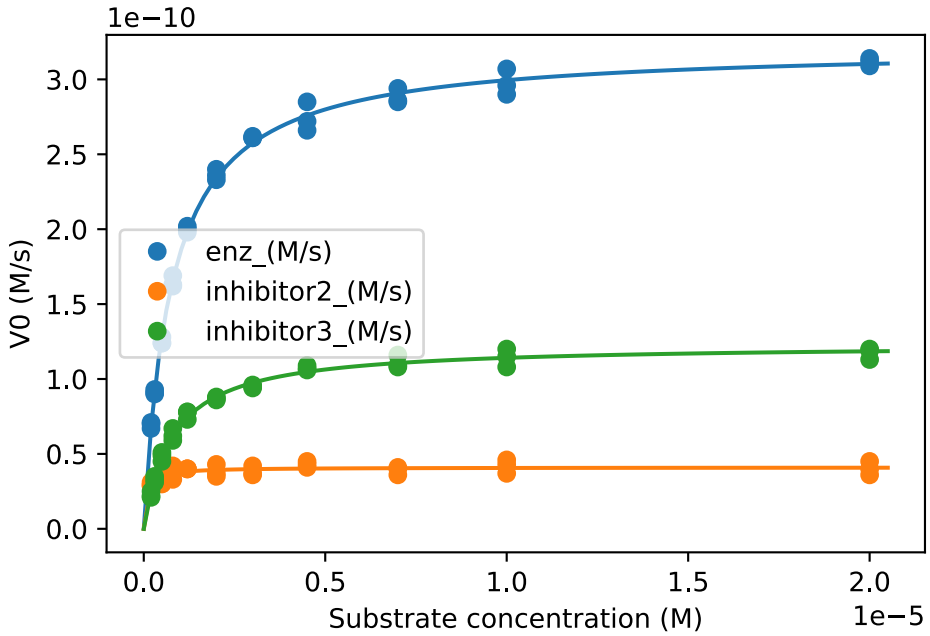
And then we can make the fit

```
## Your task fit dataset 1: 'enz_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['enz_(M/s)'],
initial_guess)
Vmax_enz_fit, K_M_enz_fit = fitted_parameters

## Your task fit dataset 2: 'inhibitor2_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['inhibitor2_(M/
s)'], initial_guess)
Vmax_inhib2_fit, K_M_inhib2_fit = fitted_parameters

## Your task fit dataset 3: 'inhibitor2_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['inhibitor3_(M/
s)'], initial_guess)
Vmax_inhib3_fit, K_M_inhib3_fit = fitted_parameters

print('Dataset 1: enz_(M/s)')
print('    V_max', Vmax_enz_fit)
print('    K_M', K_M_enz_fit)
print('Dataset 2: inhibitor2_(M/s)')
print('    V_max', Vmax_inhib2_fit)
print('    K_M', K_M_inhib2_fit)
print('Dataset 3: inhibitor3_(M/s)')
print('    V_max', Vmax_inhib3_fit)
print('    K_M', K_M_inhib3_fit)
```

```
Dataset 1: enz_(M/s)
    V_max 3.2194460122382126e-10
    K_M 7.505110277804259e-07
Dataset 2: inhibitor2_(M/s)
    V_max 4.0952644440159935e-11
```

```
    K_M 8.442553721173573e-08
Dataset 3: inhibitor3_(M/s)
    V_max 1.2316051281808638e-10
    K_M 7.913115764963332e-07
```

```
fig, ax = plt.subplots()

## Your task: Evaluate the fits
S_smooth = np.linspace(0, 2.05*10**(-5), 100)
V0_enz = michaelis_menten(S_smooth, Vmax_enz_fit, K_M_enz_fit)
V0_inhib2 = michaelis_menten(S_smooth, Vmax_inhib2_fit, K_M_inhib2_fit)
V0_inhib3 = michaelis_menten(S_smooth, Vmax_inhib3_fit, K_M_inhib3_fit)

## Plots the fits
ax.plot(S_smooth, V0_enz)
ax.plot(S_smooth, V0_inhib2)
ax.plot(S_smooth, V0_inhib3)

## Plots the data
ax.plot(df['[S]_(M)'], df['enz_(M/s)'], 'o', label='enz_(M/s)', color='C0')
ax.plot(df['[S]_(M)'], df['inhibitor2_(M/s)'], 'o', label='inhibitor2_(M/s)', color='C1')
ax.plot(df['[S]_(M)'], df['inhibitor3_(M/s)'], 'o', label='inhibitor3_(M/s)', color='C2')

## Adds legend and axis labels.
ax.legend()
ax.set_xlabel('Substrate concentration (M)')
ax.set_ylabel('V0 (M/s)')
plt.show()
```



If your fits don't closely match the data, go back and change your initial guesses.

(d) Type of inhibitor

We call $V_{\max}$ and K_{M} recorded in the presence of the inhibitor for $V_{\max}^{app}$ and K_{M}^{app} (app = apparent).

Calcuate how much each value changes in the presence of the inhibitor by dividing them by the parameters determined in the absence of an inhibitor.

```
## Your task: Calculate the ratio nof K_M^app / K_M
K_M_ratio2 = K_M_inhib2_fit / K_M_enz_fit
K_M_ratio3 = K_M_inhib3_fit / K_M_enz_fit

## Your task: Calculate the ratio nof Vmax^app / Vmax
Vmax_ratio2 = Vmax_inhib2_fit / Vmax_enz_fit
Vmax_ratio3 = Vmax_inhib3_fit / Vmax_enz_fit

## Prints the ratios
print('Inhibitor 2')
print('K_M', K_M_ratio2)
print('Vmax', Vmax_ratio2)

print('Inhibitor 3')
print('K_M', K_M_ratio3)
print('Vmax', Vmax_ratio3)
```

```
Inhibitor 2
K_M 0.11249073509474904
Vmax 0.12720401051760136
Inhibitor 3
K_M 1.0543636898135547
Vmax 0.3825518811308258
```

Based on the above, what is the likely inhibition type of inhibitor2 and inhibitor3?

! Answer

For inhibitor2 both K_M and $V_{\max}$ change significantly, making the likely type of inhibition **uncompetitive**.

For inhibitor3 only $V_{\max}$ changes significantly, making the likely type of inhibition **non-competiive**.

(e) K_i

Determine the K_i for each of the two inhibitors. The inhibitor concentration is 5 μM.

```
## Your task: Assign known value
I = 5 * 10**(-6)

## Your task: Calculate K_I for both cases.
## You should derive the correct expression before doing any coding!
K_I_inhib2 = I / (Vmax_enz_fit / Vmax_inhib2_fit - 1)
```

```
K_I_inhib3 = I / (Vmax_enz_fit / Vmax_inhib3_fit - 1)

## Prints the results
print('Inhibitor 2: ', K_I_inhib2)
print('Inhibitor 3: ', K_I_inhib3)
```

```
Inhibitor 2:  7.287155993523652e-07
Inhibitor 3:  3.0978463569655915e-06
```

! Answer

For both (pure) non-competitive and uncompetitive inhibition we have

$$V_{\max}^{app} = \frac{V_{\max}}{1 + \frac{[I]}{K_I}}.$$

From which we can isolate K_I, doing a bit of algebra we get

$$K_I = \frac{[I]}{\frac{V_{\max}}{V_{\max}^{app}} - 1}.$$

Also good practice to think about units, note that $V_{\max}$ and $V_{\max}^{app}$ have the same units ($\mathrm{M\,s^{-1}}$), so their ratio is dimensionless:

$$\frac{V_{\max}}{V_{\max}^{app}} \quad \text{units:} \quad \frac{\mathrm{M\,s^{-1}}}{\mathrm{M\,s^{-1}}} = 1.$$

Therefore the units of K_I are

$$K_I \quad \text{units:} \quad \frac{\mathrm{M}}{(1) - 1} \quad \Rightarrow \quad \mathrm{M}.$$

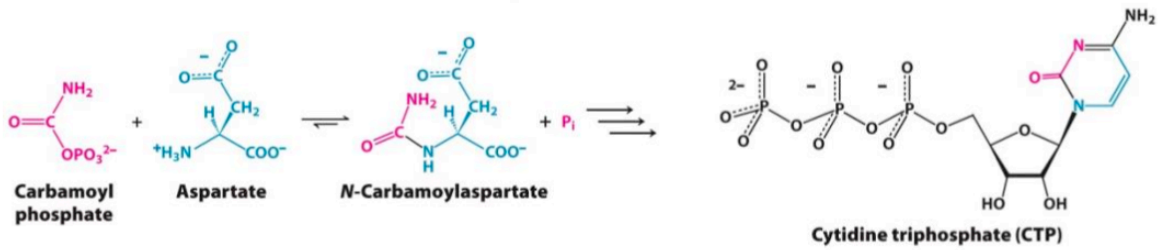
Which is the expected unit.

---

5 Enzymatic behaviour of the enzyme ATCase

```
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import numpy as np
import pandas as pd
from fysisk_biokemi.widgets import DataUploader
from IPython.display import display
```

The enzyme aspartate transcarbamoylase (ATCase) catalyzes the first reaction in the biosynthesis of pyrimidines such as CTP as shown in the reaction below:



ATCase does not obey the Michaelis-Menten kinetics model but instead shows the behaviour recorded in the `enzyme-behav-atcase.csv` dataset.

The dataset consists of three columns; the aspartate concentration in mM and the rate of formation of N-carbamyolaspartate with and without the presence of CTP.

Load the dataset with the widget below

```
uploader = DataUploader()
uploader.display()
```

Run the next cell **after** uploading the file

```
df = uploader.get_dataframe()
display(df)
```

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('atcase') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[aspartate]_(mM)	rate_(uM/s)	rate_ctp_(uM/s)
0	0.000000	−0.006903	0.005021
1	1.724138	0.330083	0.055910
2	3.448276	1.582132	0.404360
…	…	…	…

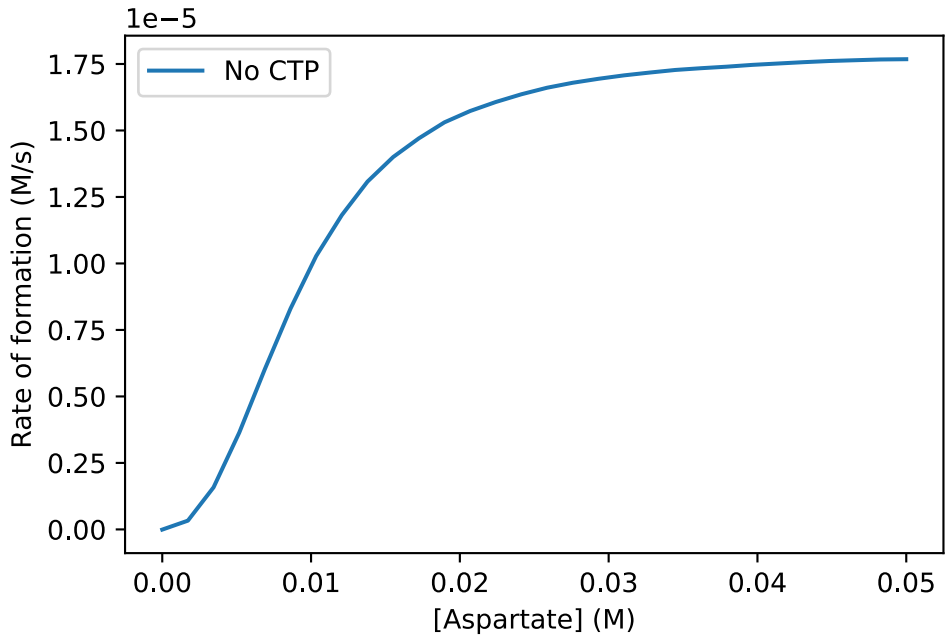
	[aspartate]_(mM)	rate_(uM/s)	rate_ctp_(uM/s)
27	46.551724	17.641138	17.347028
28	48.275862	17.669610	17.393272
29	50.000000	17.679950	17.446557

(a) Units & Plot

```
## Your task: Add columns with the properties in SI units.
df['[aspartate]_(M)'] = df['[aspartate]_(mM)'] * 10**(-3)
df['rate_(M/s)'] = df['rate_(uM/s)'] * 10**(-6)
df['rate_ctp_(M/s)'] = df['rate_ctp_(uM/s)'] * 10**(-6)
```

Make a plot of the `rate_(M/s)` dataset versus the aspartate concentration.

```
fig, ax = plt.subplots()

## Your task: Plot the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], label='No CTP')

ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



(b) Kinetic profile

Looking at the curve without CTP, describe the kinetic profile of ATCase and explain what it tells us about the way ATCase works. (You may find inspiration in the material previously covered on protein-ligand interactions)

! Answer

The midpoint has a steep increase (large slope), whereas the starting and end points has smaller slopes. In other words, the reaction velocity V_0 shows sigmoidity as a function of substrate concentration, [aspartate].

This tells us that ATCase works by cooperativity in substrate binding (i.e. does not obey Michaelis-Menten kinetics model), i.e. binding of one susbtrate increases the likelihood that the second substrate will bind (you have already encountered this in the topic of protein-ligand interactions).

(c) Quarternary structure

What does the figure tell us about the quaternary structure of ATCase?

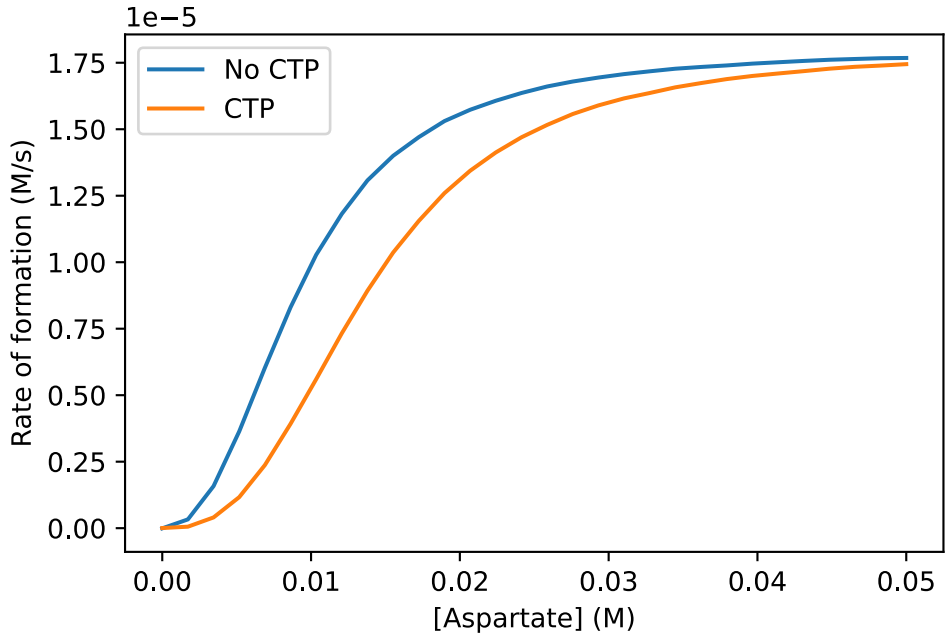
! Answer

Allosteric enzymes consist of two or more subunits, each able to bind and process a substrate molecule. Such enzymes can take at least two different conformational states, which are in equilibrium.

(d) Effect of CTP

Copy your plotting code from above and add a line to also plot the `rate_ctp_(M/s)` dataset.

```
fig, ax = plt.subplots()

## Your task: Plot the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], label='No CTP')
ax.plot(df['[aspartate]_(M)'], df['rate_ctp_(M/s)'], label='CTP')

ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



Qualitatively describe the effect of CTP on the rate of N-carbamyolaspartate formation

! Answer

CTP lowers the catalytic rate of ATCase, by binding to it and stabilising ATCase in a low substrate-affinity state.

(e) Physiological advantage

In fact many enzymes are regulated by certain end products in a fashion similar to the CTP effect on ATCase. Can you explain why this might be a physiological advantage?

! Answer

As seen from the reaction scheme, the reaction catalyzed by ATCase ultimately leads to the formation of CTP. When [CTP] increases the level of CTP formation therefore drops.

(f) Regression with Hill equation.

There is also a formulation of the Hill equation for the enzyme kinetics, which has the following form:

$$v = \frac{V_{\max} \cdot S^h}{K_{\frac{1}{2}}^h + S^h}$$

Note, that we use a formulation of the Hill equation, where the constant on the denominator is raised to the power of h. Therefore, the constant also marks the half-way saturation concentration and we denote it $K_{\frac{1}{2}}$

As usual start by writing a function implementing this equation

```
def hill_eq(S, Vmax, K, h):
    ## Your task: Implement the enzyme kinetics Hill equation.
    return (Vmax * S**h) / (K**h + S**h)

print(hill_eq(1, 1, 1, 1)) # Should give 0.5 -> (1 * 1^1) / (1**1 + 1**1) = (1 * 1) / (1 +
1) = 1/2
```

```
0.5
```

And then make the fit for each dataset of rates vs. aspartate concentration.

```
## Your task: Make the fit for the dataset without CTP
fitted_parameters_noctp, _ = curve_fit(hill_eq, df['[aspartate]_(M)'], df['rate_(M/s)'])
Vmax_fit_noctp, K_fit_noctp, h_fit_noctp = fitted_parameters_noctp

## Your task: Make the fit for the dataset with CTP
fitted_parameters_ctp, _ = curve_fit(hill_eq, df['[aspartate]_(M)'], df['rate_ctp_(M/s)'])
Vmax_fit_ctp, K_fit_ctp, h_fit_ctp = fitted_parameters_ctp
```

The next cell prints the fitted parameters for both cases.

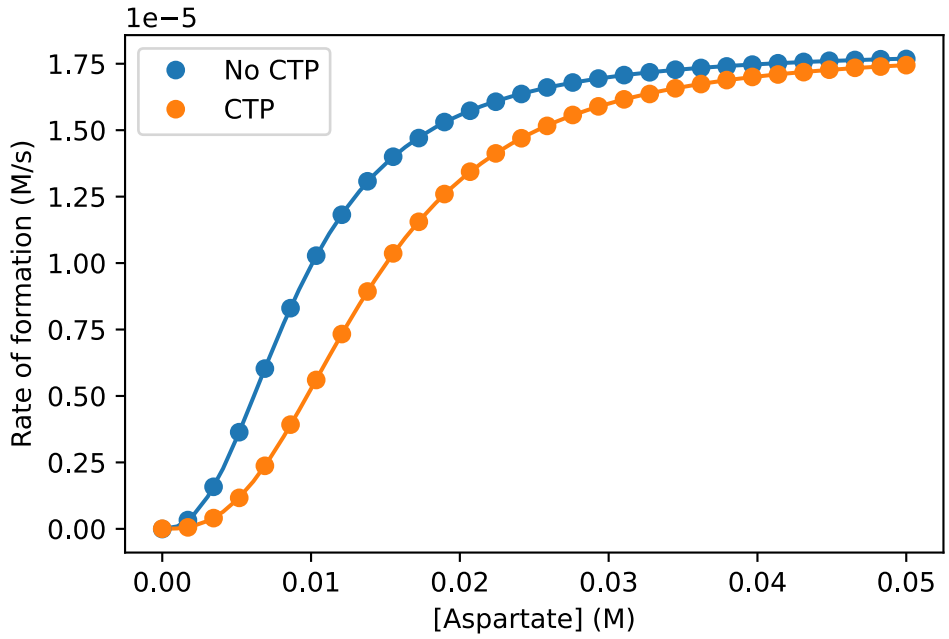
```
print('Without CTP')
print('    Vmax', Vmax_fit_noctp)
print('    K1/2', K_fit_noctp)
print('    h', h_fit_noctp)

print('With CTP')
print('    Vmax', Vmax_fit_ctp)
print('    K1/2', K_fit_ctp)
print('    h', h_fit_ctp)
```

```
Without CTP
    Vmax 1.800110730864737e-05
    K1/2 0.009189687811895926
    h 2.388780830358151
With CTP
    Vmax 1.799574252468687e-05
    K1/2 0.0138680613033327
    h 2.6985436850516282
```

Then we can plot

```
fig, ax = plt.subplots()

## Your task: Calculate the fits
S_smooth = np.linspace(0, 50*10**(-3))
V_fit_noctp = hill_eq(S_smooth, Vmax_fit_noctp, K_fit_noctp, h_fit_noctp)
V_fit_ctp = hill_eq(S_smooth, Vmax_fit_ctp, K_fit_ctp, h_fit_ctp)
```

```
## Your task: Plot the fits:
ax.plot(S_smooth, V_fit_noctp)
ax.plot(S_smooth, V_fit_ctp)

## Plots the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], 'o', label='No CTP', color='C0')
ax.plot(df['[aspartate]_(M)'], df['rate_ctp_(M/s)'], 'o', label='CTP', color='C1')

## Labels & legend
ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



How do the constants of the Hill equation change in response to the presence of CTP?
Does this match your expectations?

! Answer

V_{max} is basically unchanged.

$K_{} is increased meaning that a higher aspartate concentration is necessary in order to reach half V_{max} - this matches with the curve being shifted to the right.

h increases slightly in the presence of CTP making the curve more sigmoidal, so the switch is sharper.